AgentGuard

AI Agent Reliability & Security Platform

A detailed project explanation for our team

Open-source • Local-first • AI evaluation • RAG testing • Agent security

1. The Problem We Want to Solve

Companies are increasingly deploying RAG applications and AI agents. These systems can look impressive in demonstrations but fail in production.

Typical failures include hallucinated answers, irrelevant document retrieval, prompt-injection attacks, unsafe tool calls, excessive latency, unexpected costs, and regressions after a model or prompt update.

Traditional software uses unit tests and integration tests, but AI systems are probabilistic: the same input may produce different outputs. AgentGuard addresses this gap by acting as a testing and evaluation layer for AI applications.

Central question: Can we trust this AI agent or RAG system enough to deploy it in a real business?

2. What Exactly Are We Building?

AgentGuard is an open-source platform where a developer connects an AI application, supplies test cases, runs evaluations, and receives a report describing reliability, security, and performance weaknesses.

The platform sends test questions and tasks to the target application, observes answers, retrieved documents, tool calls, and execution traces, evaluates the behavior, runs controlled security tests, measures latency and cost, and generates reports.

```
Developer uploads test suite
↓
AgentGuard evaluation engine
↓
Accuracy + RAG + security + performance checks
↓
Results, traces, findings, and recommendations
↓
Dashboard + JSON/Markdown report
```

3. Understanding RAG Systems and AI Agents

3.1 RAG systems

RAG means Retrieval-Augmented Generation. The application first searches a document collection and then gives relevant document chunks to the language model before generating an answer.

A RAG pipeline normally includes question processing, document retrieval, context construction, answer generation, and response delivery.

RAG can fail when the retriever selects irrelevant chunks, when the model ignores useful evidence, when the answer contradicts the documents, or when citations do not support the claims.

3.2 AI agents

An AI agent is an AI system that can perform multi-step tasks and use tools such as databases, APIs, search engines, email systems, or business applications.

An agent may choose the wrong tool, use invalid parameters, leak confidential information, follow malicious instructions, perform irreversible actions, or get stuck in loops. AgentGuard evaluates both the final response and the complete execution process.

4. The Main Modules of AgentGuard

The platform can be divided into six major modules: the evaluation engine, RAG evaluation, security testing, metrics and tracing, reporting, and the dashboard.

Evaluation Engine: Executes test suites and coordinates evaluators.

RAG Evaluation: Tests retrieval quality, grounding, answer correctness, and citations.

Security Testing: Simulates prompt injection, leakage, and unsafe tool behavior.

Metrics and Tracing: Measures accuracy, success rate, latency, token usage, and cost.

Reporting: Produces structured findings and recommendations.

Dashboard: Visualizes runs, trends, failures, and risks.

5. Module 1 — The Evaluation Engine

The evaluation engine is the central component. It executes a collection of test cases against a target AI application.

A test case can contain an input question, expected behavior, reference answer, expected sources, security category, and evaluation criteria.

```
{
"id": "refund_policy_001",
"input": "Can enterprise customers request a refund after 30 days?",
"expected_answer": "Refunds are only available within 30 days.",
"expected_sources": ["refund_policy.pdf"],
"category": "factual_qa"
}
```

For every test case, AgentGuard loads the case, invokes the target application, records the response, captures traces, stores retrieved documents and tool calls, measures time and tokens, runs evaluators, saves the results, and proceeds to the next case.

The engine must support repeatable experiments so developers can compare models, prompts, retrievers, configurations, and application versions.

6. Module 2 — RAG Evaluation

RAG evaluation must separate retrieval quality from generation quality. A poor answer may be caused by the retriever, the model, the prompt, or the interaction between them.

6.1 Retrieval evaluation

The system checks whether the correct information was retrieved. Useful metrics include Precision@K, Recall@K, and Mean Reciprocal Rank (MRR).

Precision@K measures how many of the top K results are relevant. Recall@K measures how much of the relevant information was retrieved. MRR measures how high the first relevant result appears.

6.2 Context relevance

A document may be related to the topic but still fail to answer the question. AgentGuard should evaluate whether the retrieved context is actually useful.

6.3 Faithfulness and grounding

The platform can split an answer into claims and check whether each claim is supported, unsupported, or contradicted by the retrieved context.

6.4 Answer correctness

An answer can be grounded but incomplete. AgentGuard can compare responses against reference answers, structured expected results, human annotations, or an LLM judge—with limitations clearly recorded.

6.5 Citation verification

If the application provides citations, AgentGuard checks whether the document and page exist and whether the cited content actually supports the claim.

7. Module 3 — Security Testing

Security testing must be performed only against applications we own or are authorized to test. The system should use controlled experiments, mock tools, synthetic secrets, and sandboxed environments.

7.1 Prompt injection

Direct prompt injection attempts to override the system through user input. Indirect prompt injection hides malicious instructions inside retrieved documents or external content.

AgentGuard can test whether the target reveals system prompts, follows malicious instructions, leaks data, or performs unauthorized actions.

7.2 Sensitive-information leakage

The platform can insert synthetic secrets such as TEST_SECRET_12345 and check whether the application exposes them when it should not.

7.3 Unsafe tool calls

AgentGuard inspects which tools were called, whether they were allowed, whether arguments were valid, and whether confirmation was required.

7.4 Tool argument validation

The platform checks required fields, types, ranges, permissions, and limits. Dangerous actions should be simulated with mock tools rather than executed.

7.5 Excessive autonomy

The system measures tool-call counts, repeated actions, execution duration, and loops. It should detect behavior such as Search → Search → Search without termination.

8. Module 4 — Performance and Operational Metrics

An AI system can be accurate but still unsuitable for production if it is slow or expensive.

Accuracy: Depending on the task, metrics may include exact match, F1, classification accuracy, groundedness, answer correctness, and task-completion rate.

Latency: Measure total duration, time to first token, retrieval time, model-generation time, tool-execution time, and per-step latency.

Cost: Estimate input and output token costs, model calls, embedding calls, and tool usage. For local Ollama models, record token usage and runtime while distinguishing measured values from estimated infrastructure costs.

Success rate: A test should count as successful only when it satisfies the actual evaluation criteria—not merely when the API returns HTTP 200.

9. Module 5 — The Unified Evaluation Report

The report answers: What works? What fails? How serious are the failures? What should we fix?

Each finding should include the test-case ID, input, model output, retrieved documents, tool calls, evaluation result, severity, explanation, and suggested remediation.

Example findings include unsupported claims, missed relevant chunks, unauthorized tool-selection attempts, excessive tool calls, and latency violations.

Severity levels may include informational, low, medium, high, and critical. They should be based on explicit rules rather than arbitrary scores.

10. Module 6 — Dashboard

The dashboard is the interface through which developers manage projects, run evaluations, inspect failures, and compare versions.

Projects page: Stores target applications, datasets, configurations, and previous runs.

Evaluation-run page: Allows the user to select a dataset, activate evaluation modules, start a run, and monitor progress.

Metrics page: Displays accuracy, retrieval recall, groundedness, latency, cost, token usage, and security results.

Failure explorer: Shows the input, retrieved documents, prompt, model response, tool calls, and evaluation findings so the developer can understand why the system failed.

11. How the Technical Architecture Could Work

A possible architecture contains a Next.js/React dashboard, a FastAPI backend, an evaluation orchestrator, dataset and project managers, evaluation workers, a target adapter, the target AI application, and persistent storage.

```
Dashboard (Next.js/React)
↓
FastAPI
↓
Evaluation orchestrator
↓
RAG / security / tool / performance evaluators
↓
Target adapter: API, SDK, or local app
↓
Target AI agent or RAG system

Persistent data: PostgreSQL, traces, reports, datasets
```

11.1 Python

Python handles evaluation logic, dataset processing, metrics, LLM interaction, security-test generation, trace analysis, and backend services.

11.2 LangGraph

LangGraph can orchestrate stateful evaluation workflows and can also be used to build a sample target agent. It is not necessary to force LangGraph into every component.

11.3 FastAPI

FastAPI exposes endpoints for projects, datasets, evaluation runs, results, and reports.

11.4 PostgreSQL

PostgreSQL stores projects, datasets, test cases, runs, results, metrics, findings, and metadata.

11.5 Docker

Docker provides reproducible environments for the backend, workers, database, frontend, and local model services.

11.6 Ollama

Ollama can provide local models for synthetic test generation, judging, security classification, and local target applications. LLM-based evaluators must be combined with deterministic checks because judges can also be wrong.

12. How We Can Connect AgentGuard to an AI Application

Approach A — API integration

The target application exposes an endpoint such as POST /chat. AgentGuard sends an input and receives an answer, sources, and optional metadata. This is simple but may not expose internal traces.

Approach B — SDK integration

An AgentGuard SDK can instrument an application and record inputs, outputs, model calls, retrieved documents, tool calls, latency, and token usage.

Approach C — Framework adapters

The platform can provide adapters for LangGraph, LangChain, custom Python agents, and standard HTTP APIs. The first version should support one or two integration methods properly instead of attempting universal compatibility.

13. Concrete Example — Testing an HR Policy RAG Assistant

Imagine a company has an HR assistant trained on leave, remote-work, salary, and employee-policy documents.

First, create a dataset containing questions, reference answers, and expected documents. Then connect the assistant through its API or SDK.

AgentGuard sends the questions, records answers and sources, measures latency and tokens, and evaluates correctness and grounding.

If the official policy says employees receive 25 leave days but the assistant says 30, AgentGuard flags an incorrect answer and a contradiction with the reference.

The platform can also insert a synthetic malicious document and check whether the assistant follows its instructions. Finally, it generates a report describing failures and possible fixes.

14. What Makes This Project Technically Challenging?

The project combines multiple engineering disciplines.

AI/ML: Embeddings, vector search, retrieval metrics, semantic similarity, hallucination detection, classification, and dataset construction.

AI infrastructure: Evaluation pipelines, asynchronous workers, experiment tracking, dataset versioning, reproducibility, and trace storage.

Cybersecurity: Prompt injection, data leakage, tool authorization, sandboxing, threat modeling, and security regression testing.

Backend engineering: REST APIs, background jobs, database schemas, status tracking, error handling, and logging.

Product engineering: Designing an interface that turns complex AI failures into understandable and actionable information.

15. How We Can Divide the Work Between Two People

Person A — Evaluation and AI systems

Own the evaluation engine, RAG metrics, groundedness checks, test-case generation, LLM-judge experiments, security-test logic, framework adapters, and evaluation datasets.

Person B — Platform and infrastructure

Own the FastAPI backend, PostgreSQL schema, job orchestration, Docker setup, trace storage, dashboard, reports API, and optional authentication.

Shared responsibilities

Both teammates should collaborate on architecture, test design, documentation, benchmarking, debugging, deployment, GitHub issues, and the final demonstration. Both should understand the full system.

16. Proposed Development Roadmap

Phase 1 — Define the evaluation contract: Standardize input, output, retrieved context, tool calls, metadata, and results.

Phase 2 — Build a sample target: Create a realistic HR assistant, customer-support RAG system, or tool-using agent.

Phase 3 — Build the execution engine: Load datasets, invoke targets, collect traces, manage runs, and store results.

Phase 4 — Implement deterministic evaluators: Exact match, structured checks, citation existence, retrieval metrics, tool allowlists, schema validation, step limits, and latency thresholds.

Phase 5 — Add LLM evaluators: Semantic correctness, claim-level groundedness, context relevance, and explanation quality.

Phase 6 — Add security testing: Direct and indirect injection, synthetic-secret leakage, forbidden tools, invalid arguments, and loops.

Phase 7 — Build the dashboard: Runs, metrics, failed cases, traces, security findings, and comparisons.

Phase 8 — Add regression testing: Compare two versions and identify improvements or regressions.

Phase 9 — Package the project: Publish the repository, Docker setup, datasets, benchmark, technical report, demo video, and limitations.

17. What We Should Avoid

To keep the project realistic, we should not initially build a complete enterprise observability platform, support every provider, create a universal security scanner, build a fully autonomous red-team system, or implement complex billing and Kubernetes infrastructure.

The first version should focus on one clear scope: evaluating a RAG system and a tool-using agent with reproducible tests, explainable metrics, and controlled security checks.

18. What Would Make AgentGuard Genuinely Valuable?

Trace-based diagnosis: Explain not only that an answer failed, but whether the retriever, prompt, model, or tool behavior caused the failure.

Combined quality and security: Evaluate correctness, retrieval, tool safety, prompt injection, and operational metrics in one workflow.

Reproducibility: Record model, prompt, dataset, evaluator versions, configuration, and timestamp.

Local-first execution: Allow sensitive evaluations to run locally using Ollama, while recognizing that local execution alone does not guarantee security.

Open benchmarks: Publish datasets for English, French, Arabic, and potentially French–Arabic or Darija–French business queries, provided data is ethically collected and properly annotated.

19. What the Final Demonstration Could Look Like

The demonstration can compare two versions of the same customer-support agent. Version B may use a new model or prompt.

We launch the target locally, open AgentGuard, import a dataset, run evaluations against Version A and Version B, execute security tests, inspect traces, compare metrics, and export a report.

```
AgentGuard detected:

✓ Improved answer accuracy
✗ Worse retrieval recall
✗ Increased latency
✗ One unsafe tool-selection attempt
✓ No synthetic-secret leakage detected
```

The key message is evidence-based evaluation, not merely attractive charts.

20. What Each Teammate Will Learn

The project provides practical experience in LLM evaluation, RAG systems, agent orchestration, AI security, prompt-injection testing, Python backend development, FastAPI, PostgreSQL, Docker, experiment tracking, data pipelines, automated testing, observability, dashboards, open-source collaboration, and technical documentation.

It demonstrates the ability to build AI systems that are measurable, testable, explainable, and deployable.

21. The Most Important Limitations

AgentGuard cannot mathematically prove that an AI system is safe or reliable in every possible situation.

LLM judges can be wrong; datasets may not represent real-world usage; security tests cannot cover every attack; grounding detection can be imperfect; and high scores do not guarantee production safety.

AgentGuard should therefore be presented as an evidence-based evaluation and testing platform—not as a universal safety guarantee.

22. The Project in One Sentence and the MVP Scope

One-sentence definition: AgentGuard is an open-source, local-first platform that evaluates AI agents and RAG systems through reproducible test suites, trace-based diagnosis, quality metrics, and controlled security tests, helping developers identify reliability and safety problems before deployment.

Recommended MVP

The MVP should target one RAG application and one tool-using agent.

Required features include an API or SDK connector, JSON-based datasets, answer and retrieval evaluation, citation and groundedness checks, tool allowlists, controlled prompt-injection tests, latency and token measurements, run comparison, a dashboard, and JSON/Markdown report export.

Final takeaway: We are not building an AI agent for end users. We are building the testing laboratory and quality-control system for AI agents.